LI.FI

EcoFacet (v3.0.0)

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj (somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Table of Contents

1 About Researcher **1**

2 Disclaimer **1**

3 Scope **1**

4 Risk classification **1**
- 4.1 Impact . . . 1
- 4.2 Likelihood . . . 2
- 4.3 Action required for severity levels . . . 2

5 Executive Summary **2**

6 Findings **3**
- 6.1 Low Risk . . . 3
  - 6.1.1 Nominal ERC20 accounting can consume residue for non-standard tokens . . . 3
  - 6.1.2 Tron custom chain ID bypasses the same-network check . . . 3
  - 6.1.3 Mismatched final swap asset can drain the Diamond's bridge-token balance . . . 3
- 6.2 Informational . . . 4
  - 6.2.1 Test coverage gaps in the signature-binding and Tron suites . . . 4
  - 6.2.2 An unconstrained `intentDestination` can publish an unfillable intent that locks funds until `rewardDeadline` . . . 4
  - 6.2.3 Persistent maximum Portal allowances increase dependency risk . . . 5
  - 6.2.4 A decoy final transfer can satisfy the Tron receiver check . . . 5
  - 6.2.5 The EIP-712 authorization does not bind the execution mode or attribution fields . . . 5

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Valinor, Kuru, MNX, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols. He has uncovered live bugs in Ethereum, Ripple, Uniswap, Morpho, Injective, Phantom, Cosmos and received bug-bounties from those projects.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

EcoFacet	`src/Facets/EcoFacet.sol`

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

Rating	Definition
High	leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	global losses <10% or losses to only a subset of users, but still unacceptable.
Low	losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

Rating	Definition
High	almost certain to happen, easy to perform, or not easy but highly incentivized
Medium	only conditionally possible or incentivized, but still relatively likely
Low	requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Severity	Required action
Critical	Must fix as soon as possible (if already deployed)
High	Must fix (before deployment if not already deployed)
Medium	Should fix
Low	Could fix

5 Executive Summary

Over the course of 5 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

This is a differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of **8 issues** were found.

Project Summary

Project Name	**LIFI Contracts**
Project Type	Solidity Smart Contracts
Repository	lifinance/contracts
Commit	b5082a519
Audit Timeline	October 29, 2026 - October 30, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found

Critical Risk	**0**
High Risk	**0**
Medium Risk	**0**
Low Risk	**3**
Gas Optimizations	**0**
Informational	**5**
Total Issues	**8**

6 Findings

6.1 Low Risk

6.1.1 Nominal ERC20 accounting can consume residue for non-standard tokens

CONTEXT: EcoFacet.sol#L156

DESCRIPTION: The direct entrypoint requests `minAmount` from the caller and immediately creates an Eco reward for the same nominal amount. It does not measure how many tokens the Diamond actually received. With a fee-on-transfer token, the Diamond may receive `minAmount - fee` while the Portal is still authorized to pull the full `minAmount`.

If the Diamond already holds enough of that token, the shortfall is taken from the existing balance. Without a residual balance, the operation normally reverts. A fee on the subsequent Portal transfer or a negative rebase can also leave the funded reward short of the recorded amount. The impact depends on LI.FI allowing a fee-charging or rebasing token for Eco routes.

RECOMMENDATION: Restrict Eco routes to standard ERC20 tokens whose transfers preserve the requested amount. As an on-chain safeguard, measure the Diamond's balance before and after `depositAsset` and revert unless the increase equals the signed `minAmount`. Building the reward from the received amount would require that amount to be included in, or otherwise reconciled with, the backend authorization.

LI.FI: Acknowledged - the Diamond does not hold funds, so a fee-on-transfer or rebasing shortfall either reverts or at most uses stale residue that is already up for grabs.

RESEARCHER: Acknowledged.

6.1.2 Tron custom chain ID bypasses the same-network check

CONTEXT: EcoFacet.sol#L295

DESCRIPTION: Tron is represented by two IDs: LI.FI uses `1885080386571452` in `bridgeData.destinationChainId`, while Eco and the Tron runtime use `728126428`. On a Tron source chain, the generic same-network check compares the LI.FI ID with `block.chainid`, so it does not recognize them as the same chain. `_validateEcoData` then accepts the LI.FI/Eco ID pair as a valid Tron destination.

The backend must sign the payload, so an arbitrary caller cannot trigger this with altered data. However, a backend-signed Tron-to-Tron request can publish a same-chain intent, charge unnecessary solver or bridge fees, and emit cross-chain accounting events for an operation that never crossed chains.

RECOMMENDATION: Normalize the destination ID before applying the same-network check, or explicitly revert when `isTronDestination` is true and `block.chainid == ECO_CHAIN_ID_TRON`. Add a test that executes both Eco entrypoints with the Tron runtime ID and LI.FI's custom Tron destination ID.

LI.FI: Fixed in a0f867f98.

RESEARCHER: Verified fix.

6.1.3 Mismatched final swap asset can drain the Diamond's bridge-token balance

CONTEXT: EcoFacet.sol#L187

DESCRIPTION: `swapAndStartBridgeTokensViaEco` accepts the amount returned by `_depositAndSwap` without checking that the final swap asset is `_bridgeData.sendingAssetId`. `_depositAndSwap` measures the balance increase of `_swapData[last].receivingAssetId`, while the positive-slippage refund and Eco reward both spend `_bridgeData.sendingAssetId`.

The backend signature does not cover `_swapData` or `hasSourceSwaps`, so a caller can reuse a valid authorization on the swap entrypoint and declare a different final asset. For example, the caller can make `_depositAndSwap` report `B` units of native output while the Diamond already holds `B` units of the signed bridge token. Eco then sends `B - minAmount` bridge tokens to the signed refund recipient and lets the Portal pull `minAmount`. The outer refund returns the native value, leaving the pre-existing bridge-token balance drained.

RECOMMENDATION: Before calling `_depositAndSwap`, require the last swap's `receivingAssetId` to equal `_bridgeData.sendingAssetId` and revert with `InformationMismatch` otherwise. Consider also signing the execution mode or entrypoint so a direct-bridge authorization cannot be reused on the swap path.

LI.FI: Fixed in a6034c50.

RESEARCHER: Verified fix.

6.2 Informational

6.2.1 Test coverage gaps in the signature-binding and Tron suites

CONTEXT: EcoFacet.sol#L287

DESCRIPTION: The suite runs against the production Portal on a Base fork and includes real HyperCore and CCTP quote routes, but a few paths that the v3.0.0 trust model depends on are not pinned:

- The `testRevert_Tampered*` family covers 8 of the 13 signed `EcoPayload` fields. Not covered: `nonEVMReceiver`, `solanaATA`, `sendingAssetId`, `transactionId`, and `deadline`. The hashes of `nonEVMReceiver` and `solanaATA` are the only thing binding the Solana receiver path, so a refactor that dropped either from the struct hash would not fail a test.
- There is no Tron negative test for a multi-call route or a final call with a non-zero `value` or an unrelated `target` (see finding 04). The suite therefore does not document that such a route passes the on-chain check when signed.
- `SignatureExpired` and `InvalidSignature` (unauthorized signer) are only exercised through `startBridgeTokensViaEco`. One swap-entrypoint variant each would protect against a future refactor that moves `_verifySignature` after `_depositAndSwap`.
- No test asserts that `nonEVMReceiver` and `solanaATA` are ignored, but still signed, for EVM destinations.

RECOMMENDATION: Add tamper tests for the five uncovered fields, a signed decoy-route Tron test (or a reverting one if finding 04 is fixed), swap-entrypoint variants of the expired and unauthorized-signer tests, and an EVM-destination test with non-empty `nonEVMReceiver` and non-zero `solanaATA`.

LI.FI: Fixed in 9df7c6e

RESEARCHER: Verified fix.

6.2.2 An unconstrained `intentDestination` can publish an unfillable intent that locks funds until `rewardDeadline`

CONTEXT: EcoFacet.sol#L293

DESCRIPTION: For an EVM `destinationChainId`, `_validateEcoData` accepts any non-zero `intentDestination` other than Eco's Solana and Tron IDs. The facet does not check that the value is a chain where Eco's Portal is deployed, that it matches the chain the route executes on, or that `route.portal` is the Portal on that chain. Earlier revisions of this PR enforced the HyperCore `1337 -> 999` mapping and a `route.portal == PORTAL` check on-chain; both were removed in favour of the backend signature.

If the backend signs a wrong value (for example `1337` instead of `999` for a HyperCore route, or the destination chain instead of the source chain for a CCTP route), `publishAndFund` succeeds, no solver can fulfil the intent, and the user's `minAmount` sits in the vault until `rewardDeadline`. After that anyone can call `Portal.refund`, which returns the reward to `reward.creator` (`refundRecipient`). The impact is a delay bounded by `rewardDeadline`, not a loss, and an ordinary caller cannot trigger it because `intentDestination` is signed.

RECOMMENDATION: Add to the backend checklist in `docs/EcoFacet.md` that `intentDestination` must be a chain with a deployed Eco Portal and must equal the chain the route executes on, and state that a mistake is only recoverable through `Portal.refund` after `rewardDeadline`. Optionally bound `rewardDeadline` on-chain (for example a maximum offset from `block.timestamp`) so a signing error cannot lock funds for an unreasonable period.

LI.FI: Acknowledged - intentDestination is backend-signed, docs/EcoFacet.md already requires the backend to sign the chain the route executes on.

RESEARCHER: Acknowledged.

6.2.3 Persistent maximum Portal allowances increase dependency risk

CONTEXT: EcoFacet.sol#L253

DESCRIPTION: `_startBridge` uses `maxApproveERC20`, which grants the Portal a near-unlimited allowance whenever the existing allowance is too small. The allowance is not reduced after `publishAndFund` completes.

If the Portal implementation, its upgrade authority, or another allowance-consuming Portal path is compromised, it can pull any current or future balance of every token previously approved through Eco. This does not give an unprivileged caller a direct exploit against an honest Portal, and the facet is not intended to retain balances, but the standing allowance increases the impact of a dependency failure.

RECOMMENDATION: Approve only `minAmount` for each call and clear any remaining allowance after `publishAndFund`. If maximum approvals are retained for compatibility or gas reasons, document the Portal as a fully trusted dependency and ensure its upgrade and administrative controls match the value exposed by the standing allowances.

LI.FI: Acknowledged - the max approval to the bridge contract follows the same pattern as our other facets, the Diamond does not hold funds, and the Portal is already trusted with every funded reward.

RESEARCHER: Acknowledged.

6.2.4 A decoy final transfer can satisfy the Tron receiver check

CONTEXT: EcoFacet.sol#L335

DESCRIPTION: `_validateTronReceiver` checks only the final call's calldata length, `transfer(address,uint256)` selector, and recipient word. It does not verify the final call target, transfer amount, earlier calls, or the relationship between the target and `route.tokens`.

A malformed route could transfer the real payout to another address in an earlier call and finish with `transfer(expectedReceiver, 0)`. A call to an unrelated contract that accepts the same selector can also pass. The complete route is covered by the backend signature, so this requires the authorized backend to sign an inconsistent or malicious route; ordinary callers cannot modify a signed route.

RECOMMENDATION: Bind the expected Tron payout token and amount in signed data and validate the final call's target, recipient, and amount against those values. If the backend is intentionally the sole authority for these details, document that the on-chain check is only a limited consistency check and must not be treated as complete receiver enforcement.

LI.FI: Acknowledged - the full Tron route is covered by the backend signature, so a caller cannot inject a decoy call. The on-chain check is a consistency check on top of it.

RESEARCHER: Acknowledged.

6.2.5 The EIP-712 authorization does not bind the execution mode or attribution fields

CONTEXT: EcoFacet.sol#L413

DESCRIPTION: The signed `EcoPayload` covers the economic bridge data and Eco route, but it does not include `hasSourceSwaps`, the selected entrypoint, `bridge`, `integrator`, or `referrer`. A holder of a valid signature can therefore switch between the direct and swap entrypoints, provided the entrypoint modifiers are satisfied, and can change the attribution values emitted in `LiFiTransferStarted`.

Reusing a direct authorization on the swap path is an important part of the mismatched-final-asset finding. After that asset mismatch is fixed, no separate fund-loss path was identified. The string and referrer fields are documented as analytics data, so their mutability is informational unless another system treats the emitted values as authenticated.

RECOMMENDATION: Include `hasSourceSwaps` or an explicit execution-mode value in the signed payload. If a signature is intended for one function only, bind the entrypoint selector as well. Either sign the attribution fields when downstream systems rely on them or document clearly that callers may change them.

LI.FI: Acknowledged - consistent with our other signature-gated facets, the signature binds the economic and route parameters.

RESEARCHER: Acknowledged.